# Generative AI and Its Applications

## Unit 3

(UE23CS342BA4)

## QUERY EXPANSION & RE-RANKING

**Dr. Pooja Agarwal**

Dept. of CSE, PES University, Bangalore

## Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

# The Query Problem!

Original Query: "best laptop for programming"
Problems with this query:
1. Ambiguity: "best" means what? (price? performance?)
2. Missing context: What kind of programming?
3. Vocabulary gap: "laptop" vs "notebook" vs "macbook"
4. Underspecified: Budget? Screen size? Battery life?

Examples of what user might think:
"laptop with good processor for software development"
"long battery life laptop for coding on the go"
"affordable laptop for programming students"
"high-performance notebook for machine learning“

Analogy:
Query expansion is like a detective asking follow-up questions before searching for evidence. The initial clue might be vague, so you need to think of all possible interpretations.

# What is Query Expansion?!

**Query Expansion** is the process of reformulating and enriching the original user query with additional relevant terms to improve retrieval recall and precision.

Original Query: Q = "machine learning"

Expanded Query: Q' = "machine learning" OR "ML" OR

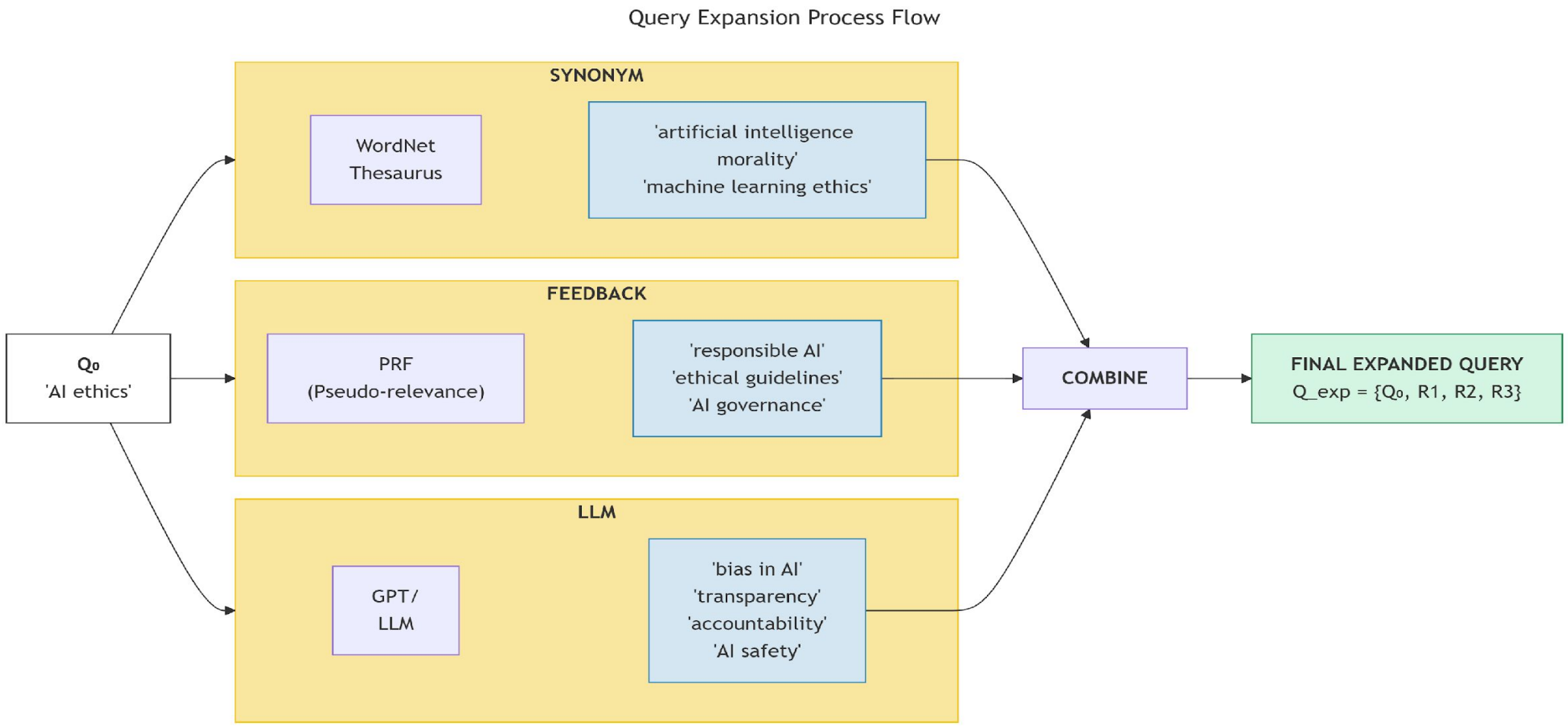
"artificial intelligence" OR "deep learning"

OR

"neural networks" OR "supervised learning"

## The Query Problem

# What Query Expansion is and Visual Representation.

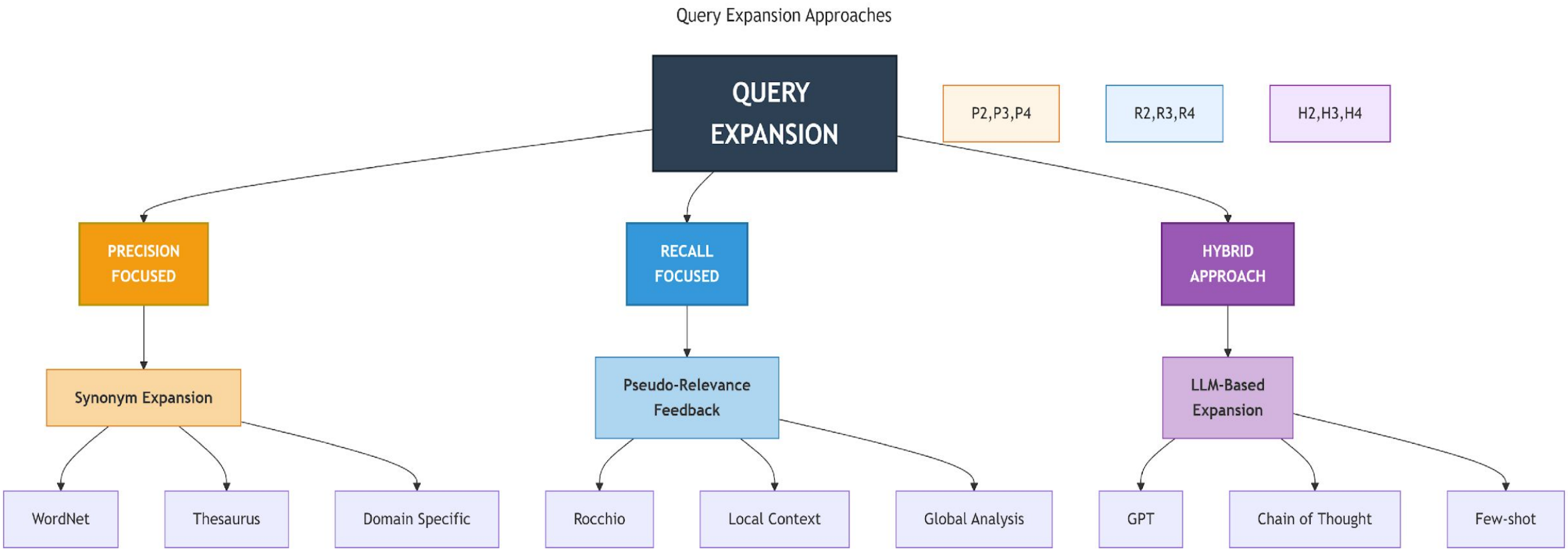


## The Query Problem

# What Query Expansion Approaches:

**Three Main Categories:**

1. **Precision-focused:** Add synonyms to catch variations
2. **Recall-focused:** Learn from initial results to add terms
3. **Hybrid:** Use LLMs to understand intent and expand intelligently



# Technique 1 - Synonym-Based Expansion.

Original Query: "automobile repair manual"

Synonym Expansion:

| Original Term | Expanded Synonyms |
|---|---|
| automobile | car, vehicle, motor vehicle |
| repair | fix, maintenance, service |
| manual | guide, handbook, instructions |

Expanded Query:
("automobile" OR "car" OR "vehicle" OR "motor vehicle") AND
("repair" OR "fix" OR "maintenance" OR "service") AND
("manual" OR "guide" OR "handbook" OR "instructions")

**Sources for Synonyms:**

- **WordNet:** Lexical database of English
- **Domain-specific thesauri:** Medical, legal, technical
- **Embedding-based:** Find nearest neighbors in vector space

# Technique 1 - Synonym-Based Expansion example.

Word: "car“

Synonyms: auto, automobile, machine, motorcar

Hyponyms: sedan, SUV, convertible, hatchback

Hypernyms: vehicle, motor vehicle, automotive

# Technique 1 - Synonym Expansion - Numerical Example.

**Problem:** Expand query and calculate new BM25 scores

**Original Query:** "car engine problem"

Doc1: "automobile engine troubleshooting guide"
Doc2: "vehicle maintenance tips"
Doc3: "motor repair handbook"
Doc4: "car engine oil change procedure"

# Technique 1 - Synonym Expansion - Numerical Example.

Step1: Synonym Expansion

| Query Term | Expanded Terms |
|---|---|
| car | car, automobile, vehicle, motor |
| engine | engine, motor |
| problem | problem, issue, trouble, troubleshooting |

Expanded query:

```
[car, automobile, vehicle, motor, engine, problem, issue, trouble, troubleshooting]
```

# Technique 1 - Synonym Expansion - Numerical Example.

Step2: BM25 Scores **Before Expansion**

Only exact matches count.

| **Document** | **Matched Terms** | **Score** |
|---|---|---|
| Doc4 | car, engine | **8.5** |
| Doc1 | engine | **5.2** |
| Doc2 | — | **0** |
| Doc3 | — | **0** |

Observations:

- Only **2 of 4 documents retrieved**
- Many relevant documents use **different vocabulary**

# Technique 1 - Synonym Expansion - Numerical Example.

**Step3: BM25 Scores After Expansion**
Now the query includes synonyms.

| Document | Matched Terms | Score | Change |
|---|---|---|---|
| Doc1 | automobile, engine, troubleshooting | **12.3** | +135% |
| Doc2 | vehicle | **7.8** | New match |
| Doc3 | motor | **6.5** | New match |
| Doc4 | car, engine | **8.5** | Same |

Why scores increase:

- More **term matches**
- BM25 increases weight when query terms appear in a document

# Technique 1 - Synonym Expansion - Numerical Example.

**Step 4: Recall Improvement**
Recall = relevant docs retrieved / total relevant docs
Before expansion:
`2 / 4 = 50%`
After expansion:
`4 / 4 = 100%`
So synonym expansion significantly improves **recall**.

Relevant documents in corpus: Doc1, Doc2, Doc3, Doc4 (4 documents)
Doc5 is irrelevant (no engine/car terms)

Recall before: 2/4 = 50% (only Doc4 retrieved)
Recall after: 4/4 = 100% (all relevant retrieved)

1. Synonym expansion improves **recall** but may sometimes reduce **precision** because:
2. Some synonyms might introduce **noise**
3. For example, "motor" could refer to **electric motors**, not car engines
4. Therefore many modern search engines (e.g., **Elasticsearch**) apply:

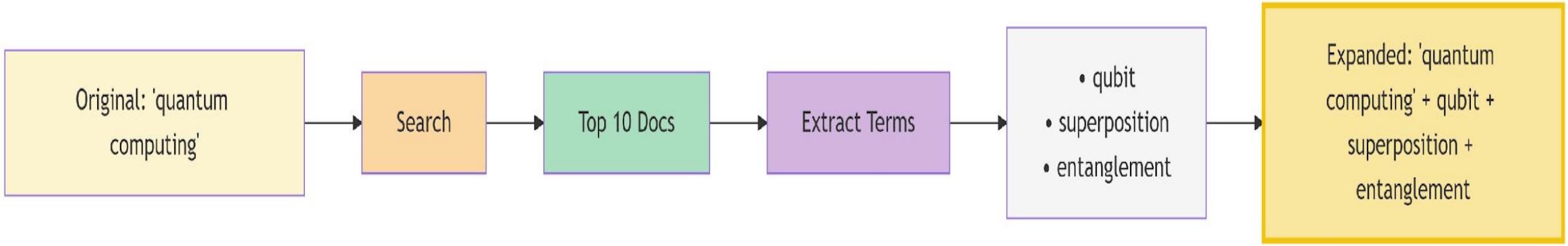
- weighted synonyms
- contextual expansion
- query-time boosting

# Technique 2 - Pseudo-Relevance Feedback (PRF).

**The Rocchio Algorithm Approach**

1. Retrieve top-k documents with original query
2. Assume these documents are relevant (pseudo-relevance)
3. Extract important terms from these documents
4. Add these terms to the original query

# Technique 2 - Pseudo-Relevance Feedback (PRF) Flow.



# Technique 2 - Pseudo-Relevance Feedback (PRF) Flow.

The **Rocchio Algorithm** is a classic method in **Information Retrieval** used for **relevance feedback** and **query expansion**. It adjusts the original query vector by moving it **closer to relevant documents** and **away from non-relevant ones**.

$$Q_new = \alpha \cdot Q_original + \beta \cdot (1/|R|) \cdot \Sigma(d \in R) - \gamma \cdot (1/|NR|) \cdot \Sigma(d \in NR)$$

| Symbol | Meaning |
|---|---|
| **Q_original** | Original query vector |
| **Q_new** | Updated query vector |
| **R** | Set of relevant documents |
| **NR** | Set of non-relevant documents |
| ** | R |
| ** | NR |
| **α (alpha)** | Weight of the original query |
| **β (beta)** | Weight for relevant documents |
| **γ (gamma)** | Penalty weight for non-relevant documents |

# Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

**Step 1: Initial Retrieval with Query "machine learning"**

Top 5 Documents:

Doc1: "machine learning algorithms for beginners"

Doc2: "deep learning neural networks tutorial"

Doc3: "supervised learning techniques"

Doc4: "python machine learning library scikit-learn"

Doc5: "artificial intelligence applications"

# Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

**Step 2: Extract Important Terms (TF-IDF based)**

| Document | Top Terms (excluding stopwords) |
|---|---|
| Doc1 | algorithms, beginners |
| Doc2 | deep, neural, networks |
| Doc3 | supervised, techniques |
| Doc4 | python, library, scikit-learn |
| Doc5 | artificial, intelligence, applications |

# Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

**Step 3: Term Frequency in Pseudo-Relevant Set)**

| Term | Frequency | Select? |
|---|---|---|
| algorithms | 1 | Yes |
| beginners | 1 | Yes |
| deep | 1 | Yes |
| neural | 1 | Yes |
| networks | 1 | Yes |
| supervised | 1 | Yes |
| python | 1 | Yes |
| scikit-learn | 1 | Yes |
| artificial | 1 | Yes |
| intelligence | 1 | Yes |

# Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

**Step 4: Expanded Query)**

Q' = "machine learning" OR algorithms OR beginners OR deep OR neural OR networks OR supervised OR python OR scikit-learn OR artificial OR intelligence

| Term | Frequency | Select? |
|---|---|---|
| algorithms | 1 | Yes |
| beginners | 1 | Yes |
| deep | 1 | Yes |
| neural | 1 | Yes |
| networks | 1 | Yes |
| supervised | 1 | Yes |
| python | 1 | Yes |
| scikit-learn | 1 | Yes |
| artificial | 1 | Yes |
| intelligence | 1 | Yes |

# Technique 2 - Numerical PRF - Weight Adjustment Example.

## Rocchio Algorithm in Action

**Initial State:** **1. Initial Query Vector**

```
q = [0.8, 0.6]
```

This represents the **original query position** in a 2-dimensional vector space.

**2. Relevant Document Centroid**

Relevant documents:

```
d1 = [0.9, 0.7]
d2 = [0.7, 0.8]
d3 = [0.8, 0.9]
```

Centroid:

```
R_centroid =
[(0.9 + 0.7 + 0.8)/3 , (0.7 + 0.8 + 0.9)/3]
= [2.4/3 , 2.4/3]
= [0.8 , 0.8]
```

This represents the **average position of relevant documents**.

# Technique 2 - Numerical PRF - Weight Adjustment Example.

## Rocchio Algorithm in Action

**Initial State:** **3. Non-Relevant Document Centroid**

Non-relevant documents:

d4 = [0.2, 0.1]

d5 = [0.3, 0.2]

Centroid:

NR_centroid =[(0.2 + 0.3)/2 , (0.1 + 0.2)/2] = [0.25 , 0.15]

This represents the **average location of irrelevant documents**.

.

# Technique 2 - Numerical PRF - Weight Adjustment Example.

**4. Apply the Rocchio Formula**

Parameters:

$\alpha = 1.0$

$\beta = 0.8$

$\gamma = 0.2$

Formula:

$q_new = \alpha q + \beta R_centroid - \gamma NR_centroid$

Substitute values:

$q_new = 1.0[0.8,0.6] + 0.8[0.8,0.8] - 0.2[0.25,0.15]$

Calculate each component:

Original query weight

$[0.8, 0.6]$

Relevant influence

$0.8 \times [0.8,0.8] = [0.64,0.64]$

Non-relevant penalty

$0.2 \times [0.25,0.15] = [0.05,0.03]$

Final vector

$q_new = [0.8,0.6] + [0.64,0.64] - [0.05,0.03] = [1.39, 1.21]$

# Technique 3 - LLM-Based Query Expansion.

**5. Normalization**

Magnitude:

$||q_new|| = \sqrt{(1.39^2 + 1.21^2)}$

$= \sqrt{(1.93 + 1.46)}$

$= \sqrt{3.39}$

$\approx 1.84$

Normalized vector:

$q_norm = [1.39/1.84 , 1.21/1.84] \approx [0.76 , 0.66]$

**6. Interpretation** The new query vector:

`[0.76 , 0.66]`

has moved:

**toward the relevant centroid** `[0.8,0.8]`

**away from the non-relevant centroid** `[0.25,0.15]`

This means the refined query will retrieve documents **more similar to relevant ones**.

**7. Geometric Intuition**

In vector space:

- Original query → somewhere between document clusters
- Relevant docs → upper region
- Non-relevant docs → lower region
- Rocchio **pulls the query toward the relevant cluster** and **pushes it away from irrelevant ones**.

# Technique 3 - LLM-Based Query Expansion

## Using Large Language Models for Intelligent Expansion

You are a query expansion expert. Given a user query, generate 5 alternative phrasings or related terms that would help retrieve relevant documents.

Original Query: {user_query}

Generate expanded queries in these categories:

- 1. Synonym replacement
- 2. More specific version
- 3. More general version
- 4. Related concepts
- 5. Common misspellings/variations

Expanded Queries:

# Technique 3 - LLM-Based Query Expansion example:

Original: "AI ethics guidelines"

LLM Output:
1. Synonym: "artificial intelligence moral principles"
2. Specific: "EU AI Act compliance requirements"
3. General: "technology ethical frameworks"
4. Related: "responsible AI development standards"
5. Variations: "AI ethic guidelines", "AI ethical guide lines"

Final Expanded Query Set:
"AI ethics guidelines" OR "artificial intelligence moral principles" OR
"EU AI Act compliance requirements" OR "technology ethical frameworks" OR
"responsible AI development standards" OR "AI ethic guidelines"

# Technique 3 - LLM Query Expansion - Chain of Thought

**Step-by-step reasoning for better expansion**

Query: "best programming language for data science"

Chain of Thought:

1. What is the user's intent?
   → Wants to learn data science, needs language recommendation

2. What are the actual data science languages?
   → Python, R, Julia, SQL

3. What factors matter for "best"?
   → Libraries (pandas, numpy, scikit-learn), community, learning curve

4. What related concepts should we include?
   → data analysis, machine learning, statistical computing, visualization

5. Generate expansion:
   Python data science
   R programming for statistics
   Julia for scientific computing
   data analysis programming languages
   machine learning libraries
   statistical computing tools
   data visualization programming

**Resulting Query:** ("programming language" OR "coding language") AND
("data science" OR "data analysis" OR "machine learning" OR
"statistics" OR "scientific computing") AND
("Python" OR "R" OR "Julia" OR "SQL")

# Query Expansion – Comparative Analysis

Effectiveness Comparison

| Technique | Precision | Recall | Computation Cost | Best For |
|---|---|---|---|---|
| **Synonym Expansion** | Medium | Medium | Very Low | General domains with known vocabulary |
| **Pseudo Relevance Feedback (PRF)** | Medium | High | Medium | Domain-specific queries with good initial results |
| **LLM-based Expansion** | High | High | High | Complex queries requiring semantic understanding |

## The Query Problem

# Techniques Example:

**Test Query**
**Query:** *"climate change effects"*

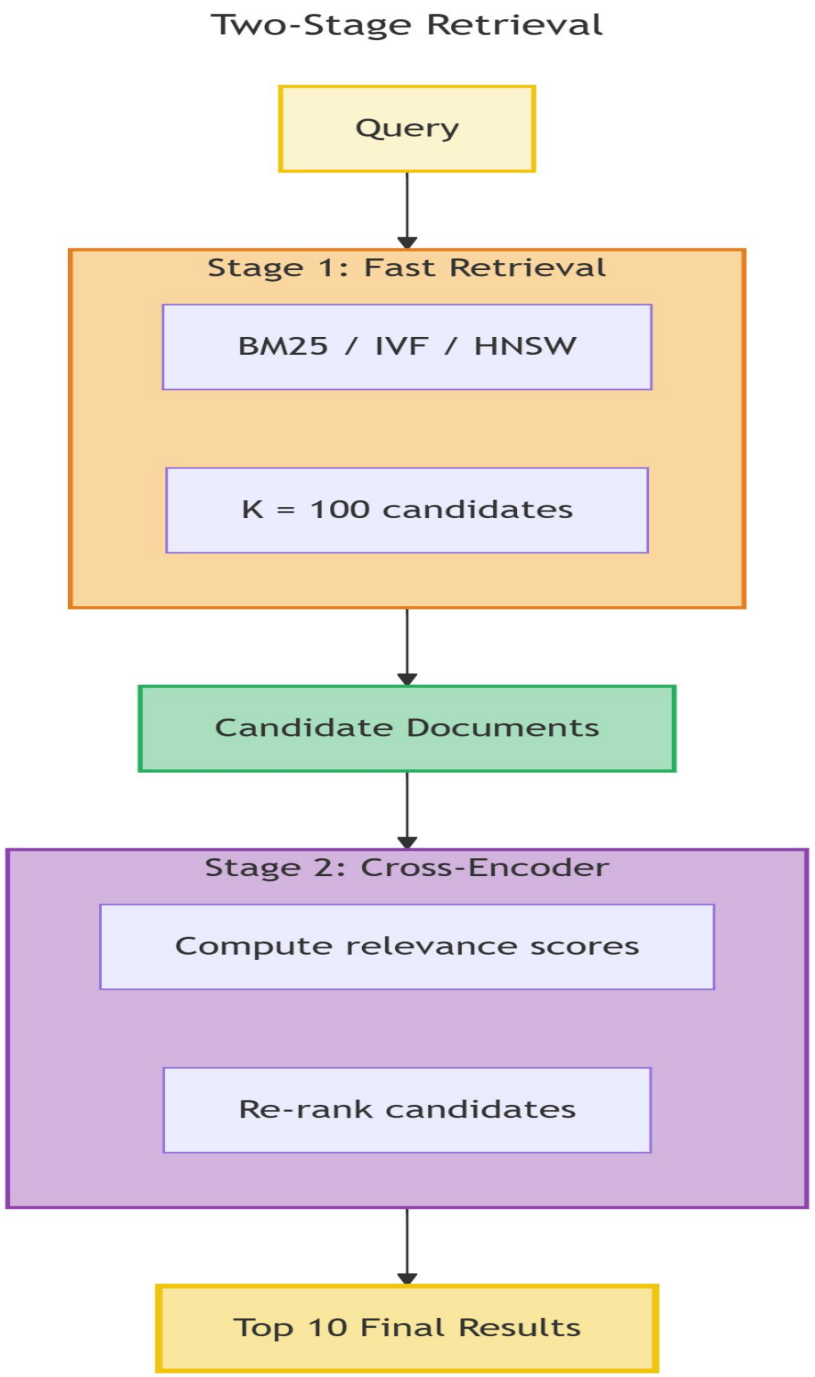
**Synonym Expansion**
Recall@10: **0.65**
Precision@10: **0.70**
Added Terms:
- global warming impacts

**Pseudo Relevance Feedback (PRF)**
Recall@10: **0.82**
Precision@10: **0.68**
Added Terms:
- sea level rise
- extreme weather

**LLM-based Expansion**
Recall@10: **0.88**
Precision@10: **0.75**
Added Terms:
- carbon emissions
- greenhouse gas
- Paris Agreement
- climate policy

# What is Re-Ranking?

**Re-ranking** is a two-stage retrieval process where an initial fast retrieval method gets top-K candidates, followed by a more accurate (but slower) model to reorder these candidates for final results.



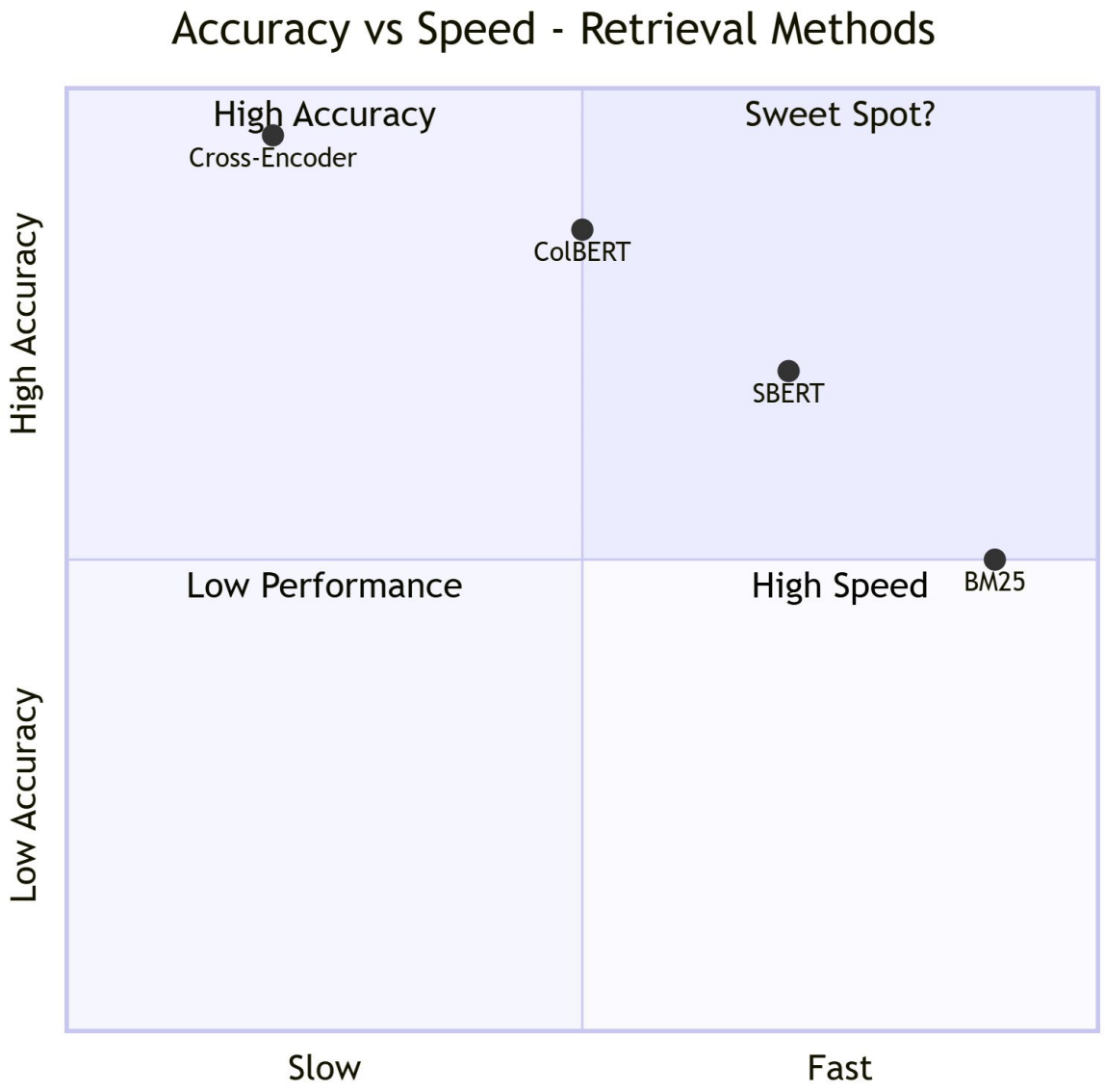
# Why Re-Ranking? The Efficiency-Accuracy Trade-off?

The Solution: Two-Stage Retrieval

Stage 1 (Fast Retrieval):
BM25 / IVF / HNSW → Retrieve top 100 candidate documents
Time: ~10 ms

Stage 2 (Accurate Re-ranking):
Cross-Encoder model → Re-rank the 100 candidates
Time: ~100 ms

Total Time ≈ 110 ms



## RE-RANKING Speed Comparison?

**Comparison:**Full corpus scan with cross-encoder:
1M documents × 50ms = 50,000,000ms = 50,000 seconds ≈ 14 hours (impossible.)
Two-stage approach:
Stage 1: 100 candidates × 0.1ms = 10ms
Stage 2: 100 candidates × 50ms = 5,000ms = 5 seconds
Total ≈ 5 seconds(possible)
Running a cross-encoder over the full corpus of 1M documents would take **~10,000 ms**, so the two-stage approach dramatically reduces computation while maintaining high accuracy.

| Method | Time for 1M Documents | Accuracy |
|---|---|---|
| BM25 only | ~10 ms | Low–Medium |
| Cross-Encoder only | ~10,000 ms | Very High |
| Two-stage Retrieval (BM25 + Cross-Encoder) | ~110 ms | High |

Two-stage retrieval systems balance **efficiency and accuracy** by:

- Using fast retrieval models to generate candidates
- Applying expensive neural models only on a small subset of documents

This architecture is widely used in modern search systems and recommendation pipelines.

# What are Re-Ranking Architectures?

**Three Main Approaches:**

## 1. Cross-Encoder Re-ranking

Query: "machine learning"
Document: "deep neural networks tutorial"

[CLS] query [SEP] document [SEP] → Transformer → [CLS] → Score

Pros: Most accurate, full interaction
Cons: Cannot pre-compute, must run for each query-doc pair

# What are Re-Ranking Architectures?

**Three Main Approaches:**

**2. Late Interaction Re-ranking (ColBERT style)**

Query tokens → [q1, q2, ...]
Document tokens → [d1, d2, ...] (pre-computed)

$$\text{Score} = \Sigma \max(q_i \cdot d_j)$$

Pros: Pre-computed doc vectors, faster than cross-encoder
Cons: Less accurate than full cross-encoder

# What are Re-Ranking Architectures?

**Three Main Approaches:**

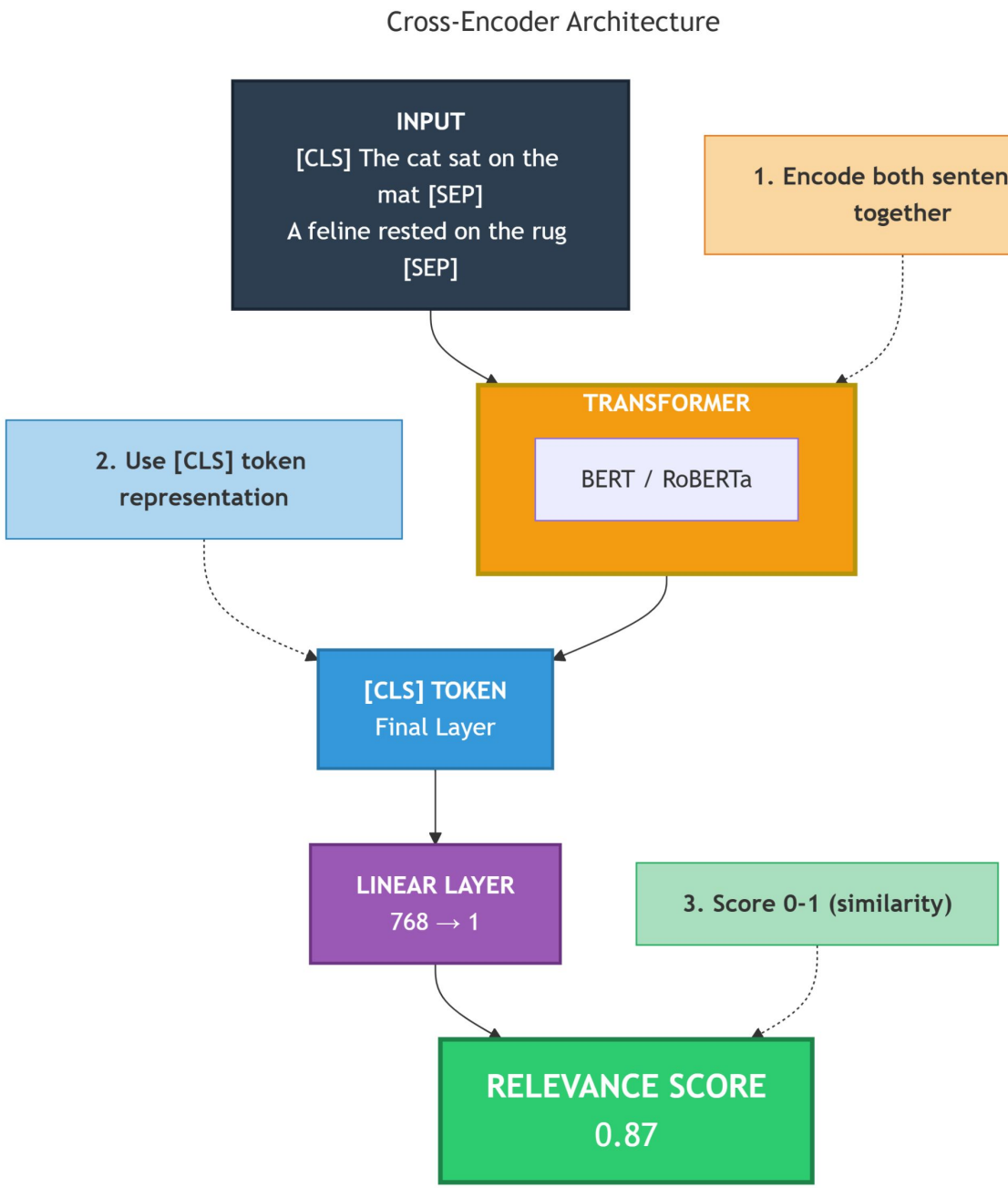
**3. Ensemble Re-ranking**

**Score = w1·BM25_score + w2·Dense_score + w3·ColBERT_score**

Pros: Combines multiple signals
Cons: Weight tuning required

# Cross-Encoder Architecture

**Key Characteristic:** Query and document interact in **every layer** through self-attention.



# Cross-Encoder Problem:

Query: "solar energy"
Document: "renewable power sources"

Token embeddings (simplified):
q_solar = [0.8, 0.3]
q_energy = [0.4, 0.7]
d_renewable = [0.7, 0.4]
d_power = [0.5, 0.6]
d_sources = [0.3, 0.2]

Cross-encoder processes ALL interactions:
Attention between q_solar and d_renewable
Attention between q_solar and d_power
Attention between q_solar and d_sources
Attention between q_energy and d_renewable
... and so on

**CALCULATION:**

## Final Score Computation

[CLS] token captures the combined representation:
h_cls = f(q_solar, q_energy, d_renewable, d_power, d_sources)
score = σ(W·h_cls + b) = 0.92

## Compare with Bi-encoder (SBERT):

q_vector = pool(q_solar, q_energy) = [0.6, 0.5]
d_vector = pool(d_renewable, d_power, d_sources) = [0.5, 0.4]
cosine = 0.98/(1.0×0.64) = 0.86

Cross-encoder (0.92) > Bi-encoder (0.86) because it captures fine-grained interactions!

# Cross-Encoder Problem:

**Step 1: Initial Retrieval (BM25)**
Query: **"quantum computing applications"**
Top documents retrieved using BM25:

| Rank | Document | BM25 Score |
|---|---|---|
| 1 | Quantum computing for machine learning | 12.5 |
| 2 | Applications of quantum mechanics | 10.2 |
| 3 | Quantum physics fundamentals | 9.8 |
| 4 | Computing in the quantum era | 9.5 |
| 5 | Machine learning algorithms | 8.7 |

# Cross-Encoder Problem:

**Step 2: Cross-Encoder Scoring**
Each query–document pair is scored by a cross-encoder model.

| Document | Cross-Encoder Score |
|---|---|
| Quantum computing for machine learning | 0.95 |
| Applications of quantum mechanics | 0.87 |
| Quantum physics fundamentals | 0.45 |
| Computing in the quantum era | 0.82 |
| Machine learning algorithms | 0.23 |

# Cross-Encoder Problem:

Step 3: Re-Rank by Cross-Encoder Score

| New Rank | Document | Cross-Encoder Score | Original BM25 Rank |
|---|---|---|---|
| 1 | Doc1 | 0.95 | 1 |
| 2 | Doc2 | 0.87 | 2 |
| 3 | Doc4 | 0.82 | 4 |
| 4 | Doc3 | 0.45 | 3 |
| 5 | Doc5 | 0.23 | 5 |

**Key Insight:**
Document 4 moved up because it is more relevant than BM25 initially estimated.
Document 3 moved down because it is less relevant.

# Cross-Encoder Problem:

Re-Ranking with MonoT5

**T5-Based Cross-Encoder for Ranking**

**Architecture:**

Input:
Query: quantum computing applications
Document: quantum computing for machine learning
Processing:
T5 Model

Output:
Classification result → **"true" or "false"**

**How MonoT5 Works:**

MonoT5 is trained to answer the question:
**“Does this document answer the query?”**

Training Format:
Input Query: {q} Document: {d}

Target:
"true" if relevant
"false" if not relevant

# Cross-Encoder Problem:

Re-Ranking with MonoT5 The model generates tokens and the probability of **"true"** is used as the relevance score.

Example1:

Input
Query: quantum computing applications
Document: quantum computing for ML

Output
"true" with probability **0.97**
Relevance Score = **0.97**

Example 2:

Input
Query: quantum computing applications
Document: quantum physics fundamentals

Output
"false" with probability **0.89**
Relevance Score = **0.11**

# MonoT5 Numerical Example:

Re-Ranking with MonoT5

**Scoring with Logits**
Vocabulary (simplified):
true, false, yes, no

**Document 1 Example**
T5 output logits:

| Token | Logit |
|---|---|
| true | 5.2 |
| false | -2.1 |
| yes | 3.1 |
| no | -1.5 |

**Softmax Calculation**
P(true) = exp(5.2) / [exp(5.2) + exp(-2.1) + exp(3.1) + exp(-1.5) + ...]
exp(5.2) = 181.27
exp(-2.1) = 0.12
exp(3.1) = 22.20
exp(-1.5) = 0.22
P(true) = 181.27 / (181.27 + 0.12 + 22.20 + 0.22)
P(true) = 181.27 / 204.56
P(true) ≈ **0.886**
Final relevance score = **0.886**

# MonoT5 Numerical Example:

## Re-Ranking with MonoT5

Ranking Multiple Documents

| Document | "true" logit | "false" logit | P(true) | Rank |
|---|---|---|---|---|
| **Doc1** | 5.2 | -2.1 | 0.886 | 2 |
| **Doc2** | 6.1 | -3.0 | 0.941 | 1 |
| **Doc3** | 2.3 | 1.2 | 0.524 | 4 |
| **Doc4** | 4.5 | -1.0 | 0.822 | 3 |

# DuoT5 – Pairwise Re-Ranking

Re-Ranking with DuoT5

**Even More Accurate: Compare Documents Directly.**

**Architecture:**

Input:
Query: q
Document A: d1
Document B: d2

Processing:
T5 Model

Output:
Prediction → **"A" or "B"** (which document is more relevant)

The model evaluates **both documents together**, allowing it to capture interactions **between documents**, not just between query and document.

**Side by side DuoT5 Working example:**
Instead of scoring each document independently, DuoT5 performs **pairwise comparisons**.

Example Input:
Query: quantum computing applications
Document A: quantum computing for machine learning
Document B: quantum physics fundamentals

Target Output:
"A" (because Document A is more relevant to the query)

**Training**
Training examples follow this format:
Input
Query: {q}
Document A: {d1}
Document B: {d2}
Target
"A" if Document A is more relevant
"B" if Document B is more relevant
The model learns to **choose the better document**.

# Re-Ranking Metrics

**How to Evaluate Re-ranking Performance**
**1. nDCG@k (Normalized Discounted Cumulative Gain)**

**nDCG@k = DCG@k / IDCG@k**

**Where:**
**DCG@k = Σ (2^rel_i - 1) / log2(i + 1)**

**IDCG@k = ideal DCG (perfect ranking)**

Example:
Query: "machine learning"
Relevance judgments: Doc1:3, Doc2:2, Doc3:1, Doc4:0

Our ranking: [Doc2(2), Doc1(3), Doc4(0), Doc3(1)]

DCG@4 = 2^2-1/log2(2) + 2^3-1/log2(3) + 2^0-1/log2(4) + 2^1-1/log2(5)
= 3/1 + 7/1.585 + 0/2 + 1/2.322
= 3 + 4.42 + 0 + 0.43 = 7.85

IDCG@4 = 2^3-1/1 + 2^2-1/1.585 + 2^1-1/2 + 2^0-1/2.322
= 7/1 + 3/1.585 + 1/2 + 0/2.322
= 7 + 1.89 + 0.5 + 0 = 9.39

nDCG@4 = 7.85/9.39 = 0.836

# Re-Ranking Metrics

**How to Evaluate Re-ranking Performance**
**2. MRR (Mean Reciprocal Rank)**

**MRR = (1/|Q|) Σ (1/rank_i)**

Example:
Query 1: first relevant at rank 2 → 1/2 = 0.5
Query 2: first relevant at rank 1 → 1/1 = 1.0
Query 3: first relevant at rank 3 → 1/3 = 0.333

MRR = (0.5 + 1.0 + 0.333)/3 = 0.611

## Re-Ranking Performance Comparison

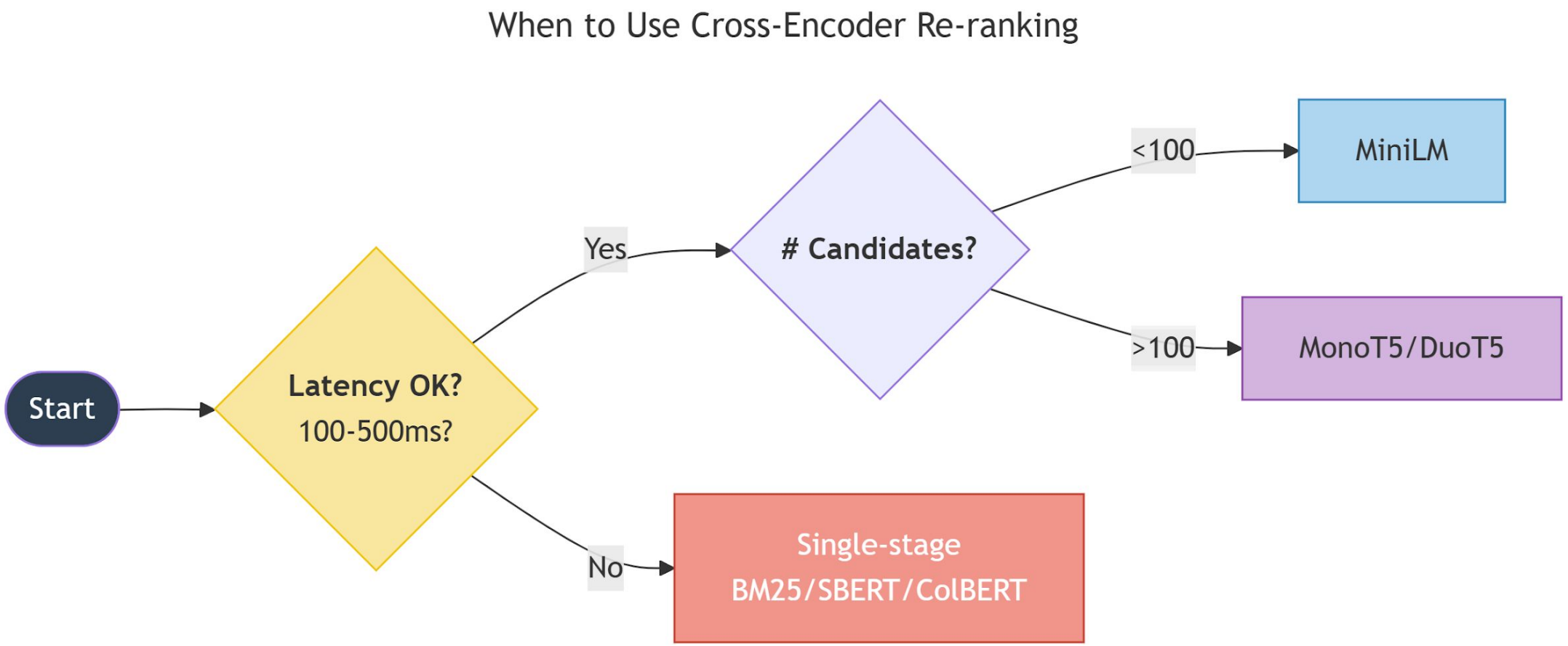
- Re-ranking adds 5-10% absolute improvement
- Trade-off: Better models are slower
- Sweet spot: MiniLM cross-encoder (good balance)

**Benchmark Results (MS MARCO Passage Ranking):**

| Model | MRR@10 | nDCG@10 | Latency (ms/100 docs) |
|---|---|---|---|
| **First Stage Retrievers** | | | |
| BM25 | 0.184 | 0.241 | 2 |
| Dense (SBERT) | 0.331 | 0.387 | 10 |
| **Second Stage Rerankers** | | | |
| ColBERT | 0.387 | 0.421 | 25 |
| MiniLM Cross-encoder | 0.408 | 0.445 | 120 |
| MonoBERT | 0.412 | 0.451 | 350 |
| MonoT5 | 0.415 | 0.455 | 400 |
| DuoT5 | 0.421 | 0.462 | 800 |

# GenerativeAI and Its Applications

**When to use what?**



## NUMERICAL PROBLEMS ON HYBRID SEARCH

### Problem 1 - Hybrid Search with BM25 + SBERT

**Problem Statement:**

You have a corpus of 5 documents. Query: **"machine learning algorithms"**

**Document Collection:**

Doc1: "Machine learning for beginners tutorial"
Doc2: "Deep learning neural networks guide"
Doc3: "Supervised learning algorithms explained"
Doc4: "Python machine learning library scikit-learn"
Doc5: "Data science with R programming"

**COMPUTE**:

1. Normalize BM25 scores (min-max normalization)
2. Normalize dense scores
3. Combine with weights α=0.4 (BM25) and β=0.6 (dense)
4. Calculate RRF scores with k=60
5. Compare weighted vs RRF rankings

## NUMERICAL PROBLEMS ON HYBRID SEARCH

**Min-Max normalization formula:**

$$BM25_{norm} = \frac{BM25 - BM25_{min}}{BM25_{max} - BM25_{min}}$$

- BM25_min = 3.4
- BM25_max = 9.1
- Range = 9.1 - 3.4 = 5.7

Normalized BM25:

| Doc | Calculation | Normalized BM25 |
|---|---|---|
| Doc1 | (8.5-3.4)/5.7 | 5.1/5.7 ≈ 0.895 |
| Doc2 | (6.2-3.4)/5.7 | 2.8/5.7 ≈ 0.491 |
| Doc3 | (9.1-3.4)/5.7 | 5.7/5.7 = 1.0 |
| Doc4 | (7.8-3.4)/5.7 | 4.4/5.7 ≈ 0.772 |
| Doc5 | (3.4-3.4)/5.7 | 0/5.7 = 0.0 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 2: Normalize Dense Scores (Cosine Similarities)

Min-max normalization formula is same:

- Min = 0.62
- Max = 0.98
- Range = 0.36

Normalized Dense:

| Doc | Calculation | Normalized Dense |
|---|---|---|
| Doc1 | (0.98-0.62)/0.36 = 0.36/0.36 | 1.0 |
| Doc2 | (0.76-0.62)/0.36 = 0.14/0.36 | 0.389 |
| Doc3 | (0.97-0.62)/0.36 = 0.35/0.36 | 0.972 |
| Doc4 | (0.95-0.62)/0.36 = 0.33/0.36 | 0.917 |
| Doc5 | (0.62-0.62)/0.36 = 0 | 0.0 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 3: Weighted Combination (α BM25 + β Dense)

Weights:

- α = 0.4
- β = 0.6

Weighted score formula:

$$Score = \alpha \cdot BM25_{norm} + \beta \cdot Dense_{norm}$$

**Weighted Ranking Order**:

1. Doc3 ⟶ 0.983
2. Doc1 ⟶ 0.958
3. Doc4 ⟶ 0.859
4. Doc2 ⟶ 0.429
5. Doc5 ⟶ 0.0

| Doc | Calculation | Weighted Score |
|---|---|---|
| Doc1 | 0.40.895 + 0.61.0 = 0.358 + 0.6 | 0.958 |
| Doc2 | 0.40.491 + 0.60.389 = 0.196 + 0.233 | 0.429 |
| Doc3 | 0.41.0 + 0.60.972 = 0.4 + 0.583 | 0.983 |
| Doc4 | 0.40.772 + 0.60.917 = 0.309 + 0.550 | 0.859 |
| Doc5 | 0.40.0 + 0.60.0 = 0 | 0.0 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 4: Reciprocal Rank Fusion (RRF)

RRF formula:

$$RRF_score = \sum_i \frac{1}{k + rank_i}$$

- Here, **k = 60** (standard small constant to dampen high ranks)
- **rank_i** = position of document in each ranking (BM25 & Dense separately)

**Step 4a: Assign ranks**

- BM25 ranking: Doc3(1), Doc1(2), Doc4(3), Doc2(4), Doc5(5)
- Dense ranking: Doc1(1), Doc3(2), Doc4(3), Doc2(4), Doc5(5)

**Step 4b: Compute RRF score for each doc**

$$RRF = \frac{1}{60 + rank_{BM25}} + \frac{1}{60 + rank_{Dense}}$$

Example for Doc1:

$$RRF_{Doc1} = \frac{1}{60+2} + \frac{1}{60+1} = 1/62 + 1/61 \approx 0.01613 + 0.01639 \approx 0.03252$$

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 4: Reciprocal Rank Fusion (RRF)

| Doc | BM25 Rank | Dense Rank | RRF |
|---|---|---|---|
| Doc1 | 2 | 1 | 0.01613 + 0.01639 = 0.03252 |
| Doc2 | 4 | 4 | 1/64 + 1/64 = 0.03125 |
| Doc3 | 1 | 2 | 1/61 + 1/62 ≈ 0.01639 + 0.01613 = 0.03252 |
| Doc4 | 3 | 3 | 1/63 + 1/63 ≈ 0.03175 |
| Doc5 | 5 | 5 | 1/65 + 1/65 ≈ 0.03077 |

**Step 4c: RRF Ranking Order**

1. Doc1 / Doc3 → tie (0.03252)
2. Doc4 → 0.03175
3. Doc2 → 0.03125
4. Doc5 → 0.03077

RRF tends to **boost documents that appear high in both rankings**.

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 5: Comparison

| Document | Weighted Score | Weighted Rank | RRF Score | RRF Rank |
|---|---|---|---|---|
| Doc1 | 0.958 | 2 | 0.0325 | 1 (tie) |
| Doc2 | 0.429 | 4 | 0.0312 | 4 |
| Doc3 | 0.983 | 1 | 0.0325 | 1 (tie) |
| Doc4 | 0.859 | 3 | 0.0318 | 3 |
| Doc5 | 0.000 | 5 | 0.0308 | 5 |

# GenerativeAI and Its Applications

## NUMERICAL PROBLEMS ON HYBRID SEARCH

**Problem 2 - Hybrid Search with Query Expansion**

**Problem Statement:**

Original Query: **"AI ethics"**

**Step 1: Apply Query Expansion using LLM**

Expanded terms:
- "artificial intelligence morality"
- "machine learning guidelines"
- "responsible AI principles"
- "AI bias fairness"
- "ethical AI frameworks"

**Step 2: Retrieve documents with expanded query**

**Documents:**

DocA: "Guidelines for ethical artificial intelligence development"
DocB: "Machine learning bias and fairness in healthcare"
DocC: "Principles of responsible innovation in tech"
DocD: "AI regulation and policy frameworks"
DocE: "Data privacy in machine learning systems"

| Document | Contains Original? | Contains Expanded? | BM25 Score | Dense Scor |
|---|---|---|---|---|
| DocA | AI, ethics | artificial intelligence, guidelines, ethical | 12.5 | 0.95 |
| DocB | none | machine learning, bias, fairness | 8.2 | 0.92 |
| DocC | none | principles, responsible | 6.7 | 0.78 |
| DocD | none | regulation, frameworks | 5.1 | 0.85 |
| DocE | none | data privacy, machine learning | 7.3 | 0.88 |

**Step 3: Compute relevance scores**

**Task:** Calculate hybrid scores (α=0.3 BM25, β=0.7 Dense) and rank documents.

## NUMERICAL PROBLEMS ON HYBRID SEARCH

**Step 1: Query Expansion (LLM)**

Expanded terms:

- artificial intelligence morality
- machine learning guidelines
- responsible AI principles
- AI bias fairness
- ethical AI frameworks

**Step 2: Retrieve Documents with Expanded Query**

| Document | Text |
|---|---|
| DocA | Guidelines for ethical artificial intelligence development |
| DocB | Machine learning bias and fairness in healthcare |
| DocC | Principles of responsible innovation in tech |
| DocD | AI regulation and policy frameworks |
| DocE | Data privacy in machine learning systems |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 3: Relevance Scores

| Document | Contains Original | Contains Expanded | BM25 | Dense (Cosine) |
|---|---|---|---|---|
| DocA | AI, ethics | artificial intelligence, guidelines, ethical | 12.5 | 0.95 |
| DocB | none | machine learning, bias, fairness | 8.2 | 0.92 |
| DocC | none | principles, responsible | 6.7 | 0.78 |
| DocD | none | regulation, frameworks | 5.1 | 0.85 |
| DocE | none | data privacy, machine learning | 7.3 | 0.88 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

**Step 4: Normalize Scores (Min-Max)**
**BM25 Normalized** (min=5.1, max=12.5):

| Doc | BM25_norm |
|---|---|
| DocA | 1.000 |
| DocB | 0.419 |
| DocC | 0.216 |
| DocD | 0.000 |
| DocE | 0.297 |

**Dense Normalized** (min=0.78, max=0.95):

| Doc | Dense_norm |
|---|---|
| DocA | 1.000 |
| DocB | 0.824 |
| DocC | 0.000 |
| DocD | 0.412 |
| DocE | 0.588 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

### Step 5: Weighted Combination (α=0.3 BM25, β=0.7 Dense)

$$Score = 0.3 * BM25_{norm} + 0.7 * Dense_{norm}$$

| Doc | Weighted Score | Calculation |
|---|---|---|
| DocA | 1.000 | 0.31 + 0.71 = 1.0 |
| DocB | 0.703 | 0.30.419 + 0.70.824 = 0.703 |
| DocC | 0.065 | 0.30.216 + 0.70 = 0.065 |
| DocD | 0.288 | 0.30 + 0.70.412 = 0.288 |
| DocE | 0.501 | 0.30.297 + 0.70.588 = 0.501 |

## NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 6: Final Ranking

| Rank | Document | Score | Title |
|---|---|---|---|
| 1 | DocA | 1.000 | Guidelines for ethical AI development |
| 2 | DocB | 0.703 | Machine learning bias and fairness in healthcare |
| 3 | DocE | 0.501 | Data privacy in machine learning systems |
| 4 | DocD | 0.288 | AI regulation and policy frameworks |
| 5 | DocC | 0.065 | Principles of responsible innovation in tech |

Query expansion and hybrid scoring significantly boosts relevant documents that do not contain the exact original query terms.

## NUMERICAL PROBLEMS ON HYBRID SEARCH Re-Ranking with Cross-Encoder

**Problem Statement:**

Initial BM25 retrieval for query **"deep learning neural networks"** returned:

| Rank | Document | BM25 Score |
|---|---|---|
| 1 | "Deep learning tutorial for beginners" | 11.2 |
| 2 | "Neural networks and deep learning book" | 10.5 |
| 3 | "Convolutional neural networks explained" | 9.8 |
| 4 | "Machine learning fundamentals" | 8.7 |
| 5 | "Deep reinforcement learning algorithms" | 8.1 |

A cross-encoder gives these relevance scores:

| Document | Cross-Encoder Score |
|---|---|
| "Deep learning tutorial for beginners" | 0.72 |
| "Neural networks and deep learning book" | 0.89 |
| "Convolutional neural networks explained" | 0.91 |
| "Machine learning fundamentals" | 0.45 |
| "Deep reinforcement learning algorithms" | 0.68 |

- Re-rank documents using cross-encoder scores
- Calculate nDCG@5 before and after re-ranking (assume relevance judgments: Doc3=3, Doc2=2, Doc1=2, Doc5=1, Doc4=0). Compute improvement percentage?

## NUMERICAL PROBLEMS ON HYBRID SEARCH Re-Ranking with Cross-Encoder

Step 1: Re-rank Documents

| New Rank | Document | Cross-Encoder Score | Original BM25 Rank |
|---|---|---|---|
| 1 | Convolutional neural networks explained | 0.91 | 3 ⬆️ |
| 2 | Neural networks and deep learning book | 0.89 | 2 |
| 3 | Deep learning tutorial for beginners | 0.72 | 1 ⬇️ |
| 4 | Deep reinforcement learning algorithms | 0.68 | 5 ⬆️ |
| 5 | Machine learning fundamentals | 0.45 | 4 ⬇️ |

**Insight:**
Most relevant document (Doc3) moved from rank 3 → 1.

## NUMERICAL PROBLEMS ON HYBRID SEARCH Re-Ranking with Cross-Encoder

**Step 2: nDCG@5 for BM25 Ranking**
**Relevance Labels:**

| Document | Label |
|---|---|
| Doc3 | 3 |
| Doc2 | 2 |
| Doc1 | 2 |
| Doc5 | 1 |
| Doc4 | 0 |

BM25 Ranking: `[Doc1(2), Doc2(2), Doc3(3), Doc4(0), Doc5(1)]`

## NUMERICAL PROBLEMS ON HYBRID SEARCH Re-Ranking with Cross-Encoder

**DCG@5 formula:**

$$DCG = \sum_{i=1}^{5} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

Calculation:

$$DCG_{BM25} = \frac{2^2 - 1}{log_2 2} + \frac{2^2 - 1}{log_2 3} + \frac{2^3 - 1}{log_2 4} + \frac{2^0 - 1}{log_2 5} + \frac{2^1 - 1}{log_2 6}$$

$$DCG_{BM25} = 3/1 + 3/1.585 + 7/2 + 0 + 1/2.585 \approx 8.777$$

**IDCG@5** (ideal DCG):

$$IDCG = 7/1 + 3/1.585 + 3/2 + 1/2.322 + 0 \approx 10.82$$

$$nDCG@5(BM25) = 8.777/10.82 \approx 0.811$$

## NUMERICAL PROBLEMS ON HYBRID SEARCH Re-Ranking with Cross-Encoder

**Step 3: nDCG@5 for Re-ranked Results**

Re-ranked: `[Doc3(3), Doc2(2), Doc1(2), Doc5(1), Doc4(0)]`

$$DCG_{Re-rank} = 7/1 + 3/1.585 + 3/2 + 1/2.322 + 0 \approx 10.82$$

$$nDCG@5(Re-rank) = 10.82/10.82 = 1.000$$

**Step 4: Improvement**

$$Improvement = \frac{1.000 - 0.811}{0.811} \times 100\% \approx 23.3\%$$

Re-ranking **moved the most relevant document to the top**, achieving perfect ranking and increasing nDCG by 23.3%.

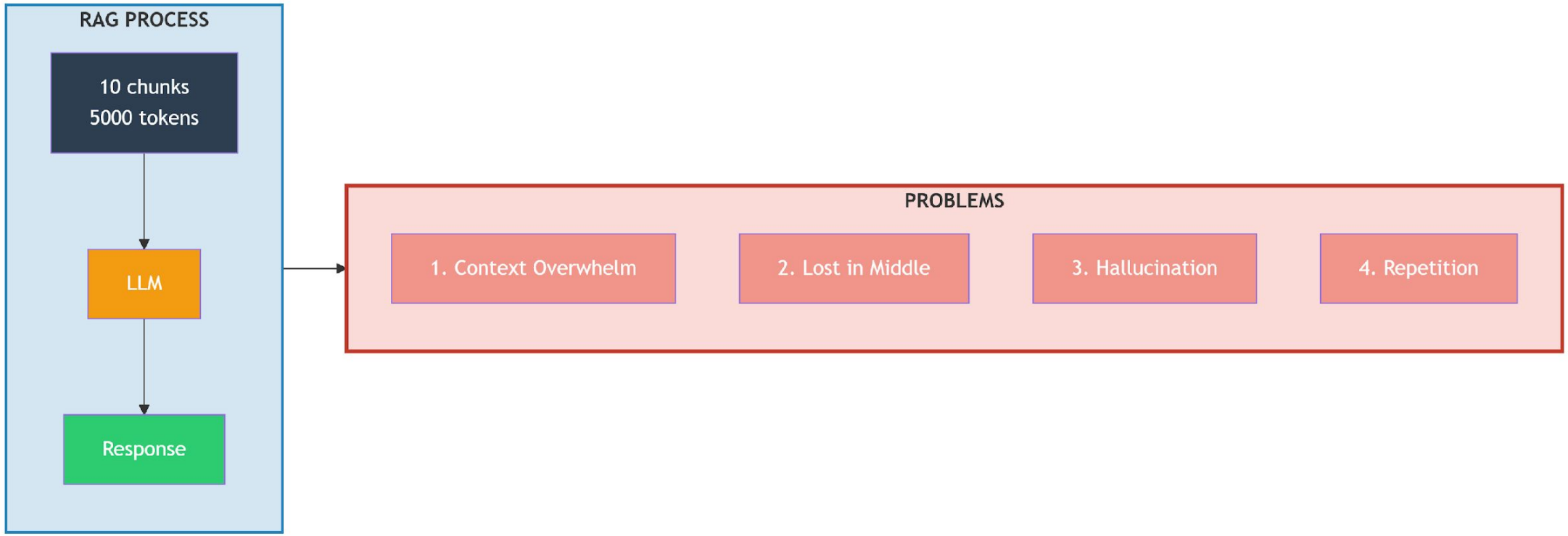
## GENERATION ENHANCEMENT TECHNIQUES

Problems:
1. LLM gets overwhelmed by too much context
2. LLM ignores middle chunks (lost in the middle)
3. LLM hallucinates when context is insufficient
4. LLM repeats information from multiple chunks

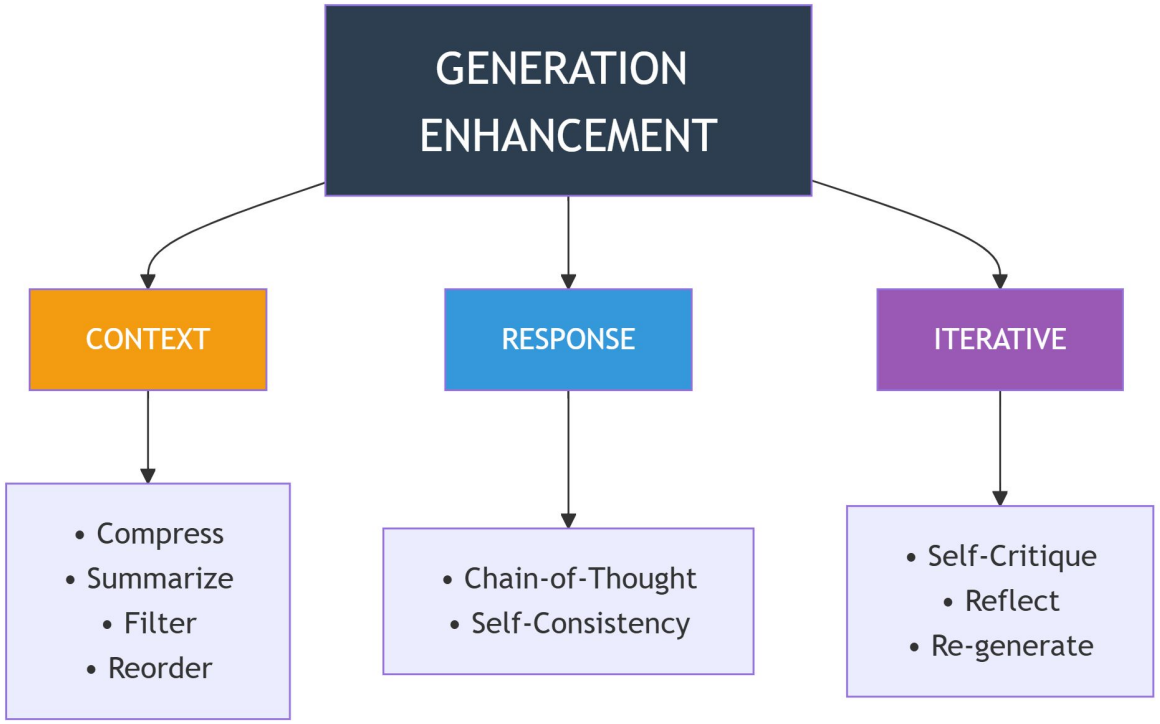
### What is Generation Enhancement?

**Generation Enhancement** refers to techniques applied after retrieval to improve how the LLM uses retrieved context to generate better, more accurate responses.

RAG Failure Modes



## GENERATION ENHANCEMENT TECHNIQUES




**Three Main Categories:**

1. **Context Optimization:** Make retrieved info more usable
2. **Response Synthesis:** Better prompting strategies
3. **Iterative Refinement:** Generate, evaluate, improve

## GENERATION ENHANCEMENT TECHNIQUES

### Technique 1 - Context Compression

**Problem:** Too many tokens = expensive + confusing

**Solution 1: Summarization**

text

Before: 5 chunks, 2500 tokens

Chunk 1: "Quantum computing uses qubits which can exist in superposition..."

Chunk 2: "Superposition allows qubits to be 0 and 1 simultaneously..."

Chunk 3: "Entanglement is another key principle where qubits are correlated..."

Chunk 4: "Quantum gates manipulate qubits to perform computations..."

Chunk 5: "Shor's algorithm uses quantum effects to factor large numbers..."

After Summarization (LLM-generated)

"Quantum computing leverages qubit properties like superposition and entanglement, manipulated via quantum gates, to perform computations such as Shor's factoring algorithm." (150 tokens)

Compression ratio: 2500 → 150 (16.7x reduction!)

**Solution 2: Extractive Compression**

text

"Keep only the most relevant sentences base

Original: [sent1, sent2, sent3, sent4, sent5,

Score each: [0.9, 0.2, 0.8, 0.1, 0.7, ...]

Keep top-K: [sent1, sent3, sent5]

## GENERATION ENHANCEMENT TECHNIQUES

### Technique 1 - Context Compression

After Summarization (LLM-generated):

"Quantum computing leverages qubit properties like superposition and entanglement, manipulated via quantum gates, to perform computations such as Shor's factoring algorithm." (150 tokens)

Compression ratio: 2500 → 150 (16.7x reduction!)

### Solution 2: Extractive Compression

Keep only the most relevant sentences based on query similarity:

Original: [sent1, sent2, sent3, sent4, sent5, ...]

Score each: [0.9, 0.2, 0.8, 0.1, 0.7, ...]

Keep top-K: [sent1, sent3, sent5]

GENERATION ENHANCEMENT TECHNIQUES

## Technique 2 - Context Reordering

**The "Lost in the Middle" Problem:**

Research shows LLMs pay most attention to the **beginning** and **end** of context, ignoring the middle.

**Solution: Put most relevant chunks at beginning and end!**

Before (random order):

[Chunk3 (rel=0.5), Chunk1 (rel=0.9), Chunk5 (rel=0.3),

Chunk2 (rel=0.8), Chunk4 (rel=0.4)]

After (reordered by relevance, with best at ends):

[Chunk1 (0.9), Chunk3 (0.5), Chunk4 (0.4), Chunk5 (0.3), Chunk2 (0.8)]

Best at start  Worst in middle  2nd best at end

**Improvement: 15-20% better answer quality!**

## GENERATION ENHANCEMENT TECHNIQUES

### Technique 3 - Chain-of-Thought with Context

**Standard Prompt:**

Context: {retrieved_chunks}

Question: {query}

Answer:

**Enhanced with Chain-of-Thought:**

Context: {retrieved_chunks}

Question: {query}

1. First, what are the key pieces of information from the context?

   [LLM extracts relevant facts]

2. How do these pieces relate to each other?

   [LLM identifies connections]

3. What additional reasoning is needed?

   [LLM applies logic]

4. Based on the above, the answer is:

   [LLM provides final answer]

## GENERATION ENHANCEMENT TECHNIQUES

EXAMPLE: Context: Articles about climate change effects
Question: How will rising temperatures affect crop yields?

1. Key information:
   - Temperature increases of 2°C reduce wheat yields by 6%
   - Corn yields drop 7% per degree above 30°C
   - CO2 fertilization could boost some crops by 15%

2. Relationships:
   - Temperature effect is negative and non-linear
   - CO2 effect partially offsets but varies by crop

3. Reasoning:
   - Net effect depends on region and crop type
   - Tropical regions will see more negative impacts

4. Answer: Rising temperatures will generally decrease crop yields, with impacts varying by region and crop...

## GENERATION ENHANCEMENT TECHNIQUES

### Technique 4 - Self-Consistency and Reflection

**Self-Consistency: Generate multiple answers and vote**

Round 1: Generate answer with temperature=0.7

Round 2: Generate answer with temperature=0.7

Round 3: Generate answer with temperature=0.7

Answers:

1. "The capital of France is Paris"

2. "Paris is the capital city of France"

3. "France's capital is Paris“

Consensus: Paris (3/3 votes)

**Self-Reflection: Generate, critique, improve**

Step 1: Generate initial answer

Step 2: Ask LLM to critique its own answer

Step 3: Generate improved answer based on critique

**Example:**

Initial: "Quantum computing is faster than classical computing.

Critique: "Vague - faster at what? Need to specify task types."

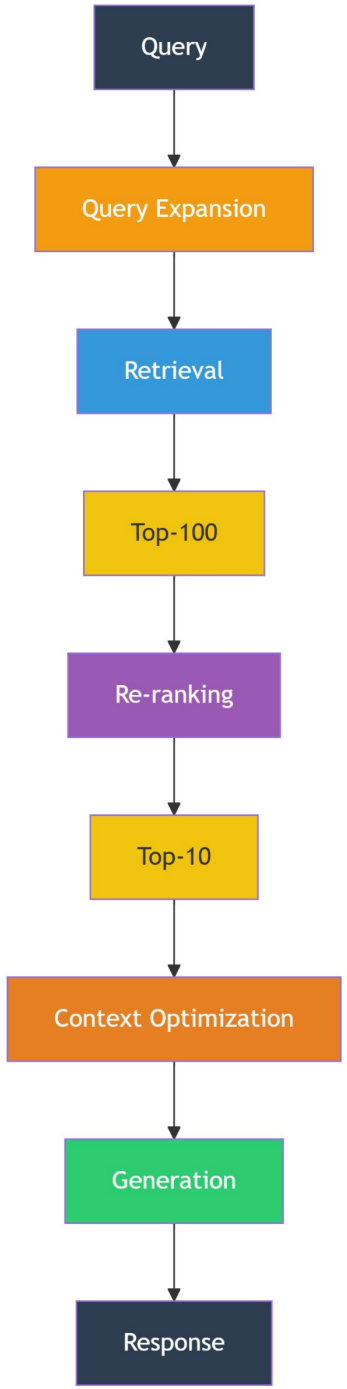
Improved: "Quantum computing provides exponential speedup

specific tasks like factoring large numbers, but is not

universally faster than classical computing."

# GenerativeAI and Its Applications

## Complete RAG Pipeline with All Enhancements



# THANK YOU